Segmentación de Clientes mediante K-Means y Análisis de Componentes Principales: Proyecto 02

Fernando Álvarez Ruiz — A01286754,
Laurie Camila Hernández Pacheco — A01286569, y
Emilio Alejandro González Huerta — A01286440

Instituto Tecnológico y de Estudios Superiores de Monterrey y
Dr. Hugo Terashima Marín
(Dated: 2 de mayo de 2026)

ABSTRACT: En el presente trabajo se implementa desde cero el algoritmo de K-means con el objetivo de comprender su comportamiento interno y evaluar su desempeño sobre un problema real de segmentación de clientes sin depender de herramientas preconstruidas. El dataset utilizado corresponde a un conjunto de 2,216 registros de clientes con variables demográficas y de consumo, sobre el cual se realizó un EDA de características, limpieza de datos y estandarización previa al entrenamiento. Se entrenaron modelos para valores de k entre 2 y 15, determinando el número óptimo de clusters mediante la técnica del codo y el coeficiente de silueta, obteniendo $k = 3$ como valor más adecuado según este último criterio, con un coeficiente aproximado de 0.68. Dado que los datos cuentan con más de tres dimensiones, se aplicó PCA para reducir la información a dos y tres componentes principales, permitiendo la visualización de los clusters resultantes. Los resultados del modelo manual fueron contrastados contra la implementación estándar de scikit-learn, observándose una convergencia comparable entre ambos con diferencias menores en puntos críticos. Estos hallazgos refuerzan que una implementación manual bien fundamentada es capaz de alcanzar un desempeño equivalente al de librerías optimizadas, y que comprender el proceso desde el preprocesamiento hasta la interpretación de los segmentos es indispensable para aplicar estas técnicas con criterio correcto.

CONTENTS

I. Introducción 1
II. Planteamiento del Problema 2
III. Objetivo del Trabajo 2
IV. Metodología y Enfoque de Solución 2
A. Base de Datos y EDA 2
1. Augmentación y Estandarización de Datos 2
B. Implementación Manual 3
1. Método del Codo 3
2. Coeficiente de la Silueta 4
C. Comparación Scikit-Learn 4
D. Implementación de PCA 5
1. Resultados de Scikit-Learn 6
E. Experimentos 6
V. Discusión 7
VI. Conclusiones 7
Referencias 8

I. INTRODUCCIÓN

Desde sus primeras formulaciones teóricas en la segunda mitad del siglo XX, la Inteligencia Artificial ha atravesado una transformación, comenzando como un conjunto de instrucciones predefinidas a gradualmente en sistemas capaces de aprender a partir de la experiencia. Machine Learning es precisamente la rama que hace posible este salto, permitiendo a las máquinas de la habilidad de descubrir estructuras y regularidades en grandes volúmenes de información sin que un programador tenga que especificar explícitamente cada regla de manera manual. Esto ha llevado a la tecnología de la simple ejecución de tareas repetitivas hacia la construcción de modelos que se adaptan y refinan conforme procesan más datos. [1]

Dentro de este enfoque, el algoritmo de K-means se encuentra como uno de los métodos de agrupamiento más empleados tanto en investigación como en aplicaciones en la industria. Dado un conjunto de datos y un número K de grupos, el algoritmo asigna de forma iterativa cada punto al centroide más próximo y recalcula dichos centroides hasta llegar a convergencia. A pesar de su simplicidad, K-means es capaz de encontrar estructuras significativas incluso en datos de alta dimensionalidad (añadiendo PCA), encontrando uso en áreas como segmentación de clientes, detección de anomalías, compresión de imágenes y sistemas de recomendación, etc. [2]

La relevancia práctica de este método sumada a la importancia de entender su comportamiento fuera del uso de librerías optimizadas en Python motiva el presente trabajo. Se implementa K-means desde cero sobre un dataset de segmentación de clientes de múltiples dimensiones, incluyendo la determinación del número óptimo de clusters mediante la técnica del codo y silueta, la reducción de dimensionalidad a través de PCA para su visualización, y la comparación contra la implementación de scikit-learn. Con esto el proyecto busca mostrar una comprensión integral del flujo de un problema de clustering desde el preprocesamiento y construcción de features, hasta la evaluación e interpretación de los segmentos resultantes.

II. PLANTEAMIENTO DEL PROBLEMA

En el ámbito del aprendizaje automático, una de las tareas más frecuentes consiste en encontrar agrupaciones naturales dentro de un conjunto de datos, sin contar con etiquetas que faciliten el proceso. Este tipo de problemas de clustering tienen especial relevancia cuando se trabaja con datos del mundo real donde la estructura subyacente no es evidente. En este contexto, el algoritmo de K-means representa una de las aproximaciones más directas y ampliamente utilizadas para abordar dicha tarea. [3]

A pesar de que herramientas como scikit-learn ofrecen implementaciones listas para usar, su aplicación directa oculta los mecanismos internos del algoritmo, como la inicialización y procesamiento de centroides, el criterio de convergencia, el cálculo de distancias y la sensibilidad ante distintos valores de K. Uitlizar este algoritmo sin comprender estos fundamentos limita nuestra capacidad de diagnosticar resultados, seleccionar hiperparámetros de forma razonada o adaptar el método a situaciones particulares.

El problema central de este proyecto yace entonces en construir una implementación propia de K-means y aplicarla sobre un dataset de segmentación de clientes con múltiples dimensiones. Esto implica no solo programar el algoritmo desde sus bases, sino también determinar el número adecuado de clusters mediante criterios objetivos como el método del codo y el coeficiente de silueta, reducir la dimensionalidad con PCA para poder visualizar los resultados, y contrastar el desempeño obtenido frente a la versión de scikit-learn. La justificación de este trabajo nace en que comprender holísticamente el flujo completo de un problema de naturaleza no supervisado, desde la preparación de los datos hasta la interpretación de los segmentos, es esencial para aplicar estas técnicas con criterios.

III. OBJETIVO DEL TRABAJO

Dicho lo anterior, el objetivo central de este proyecto es implementar el algoritmo de K-medias desde cero y aplicarlo sobre un conjunto de datos de segmentación de clientes, con la finalidad de comprender su funcionamiento interno y evaluar su desempeño en un problema real de agrupamiento no supervisado. Para ello, se busca determinar el número óptimo de clusters mediante las técnicas del codo y la silueta, reducir la dimensionalidad de los datos a través de PCA con el propósito de facilitar su visualización en menores dimensiones, y contrastar los resultados obtenidos con la implementación estándar de scikit-learn.

IV. METODOLOGÍA Y ENFOQUE DE SOLUCIÓN

A. Base de Datos y EDA

El conjunto de datos utilizado corresponde a customer_segmentation.csv, un dataset orientado al análisis de personalidad de clientes que contiene información demográfica, hábitos de consumo y respuesta a campañas de marketing. En su estado original cuenta con 2,240 registros y 29 columnas, abarcando variables de distintas naturalezas, sean numéricas continuas como *Income* y *Recency*, enteras como *Year_Birth* y los montos gastados por categoría de producto, categóricas como *Education* y *Marital_Status*, y binarias que indican la aceptación de campañas publicitarias y la presencia de quejas.

Como primer paso se identificaron los valores faltantes del dataset. La única columna con nulos fue *Income* con 24 registros ausentes. Dado el bajo porcentaje, se optó por eliminar directamente dichas filas. Adicionalmente, se removieron posibles duplicados para garantizar la integridad del conjunto. Tras este proceso, el dataset quedó conformado por 2,216 registros para el análisis.

1. *Augmentación y Estandarización de Datos*

Dada la alta dimensionalidad del dataset original, se construyeron tres nuevas variables con el objetivo de condensar información relevante y mejorar la capacidad del modelo para segmentar a los clientes

- *Total_Spending*: suma de los montos gastados en vinos, frutas, carnes, pescado, dulces y productos de lujo, capturando el gasto total del cliente.
- *Children*: combinación de Kidhome y Teenhome, representando el número total de menores en el hogar.

- *Age*: calculada como la diferencia entre el año de registro del cliente (Dt_Customer) y su año de nacimiento, aproximando la edad al momento de incorporarse a la base de datos.

Durante el análisis de estas variables se detectaron valores atípicos en Age, con registros que alcanzaban hasta 121 años, producto de errores en los datos originales. Para mitigar su impacto se aplicó un filtrado por percentiles, conservando únicamente los registros dentro del rango del percentil 1 al 99.

Las variables finales seleccionadas para el entrenamiento del modelo fueron: *Age*, *Income*, *Children*, *Recency*, *Total_Spending* y *NumDealsPurchases*. Esta selección busca capturar tanto el perfil demográfico del cliente como sus patrones de consumo y comportamiento de compra. Dado que K-means funciona con distancias euclidianas, todas las features fueron estandarizadas con StandardScaler para evitar que variables de mayor escala dominen.

Como variable de referencia para la evaluación posterior se utilizó Education, recodificada en tres categorías

- Low Education (Basic y 2n Cycle)
- Higher Education (Graduation)
- Postgraduate (Master y PhD)

B. Implementación Manual

Ya teniendo la base de datos limpia y lista para usarse, se pueden empezar a aplicar las técnicas de clustering. Para simular el funcionamiento del método K-means lo más cercano posible se ejecutaron todas las partes importantes del modelo real siguiendo el diseño algorítmico original.

El algoritmo principal que utiliza la libreía de scikit está basado en iteraciones de los mismos pasos. Inicialmente se toman puntos aleatorios en el espacio como una forma de representar centroides. Posteriormente se calcula la distancia euclidiana entre todos los puntos y cada uno de los centroides con el fin de asignarle un grupo a cada uno de los puntos. La distancia euclideana se calcula con la siguiente fórmula:

$$d(p,q) = \sqrt{\sum (p_i - q_i)^2}$$

Donde p y q son dos puntos.

Con los grupos ya armados, se pueden encontrar nuevos centroides que sean más representativos de cada grupo calculando las coordenadas promedio de cada uno de los puntos que lo conforman. De esta manera, se han creado nuevos centroides sobre los que se puede volver a repetir este proceso hasta que el movimiento de los centroides sea mínimo.

Para aplicar esto con lineas propias de código se diseñó una clase de K-means manual con funciones que pudieran realizar todas las fases del proceso. En la función *predict* se inicia generando puntos aleatorios con una función típica para valores aleatorios y luego se guardan como centroides. A partir de esto se puede empezar a iterar el proceso con la función *create_clusters*, que asigna todos los puntos al cluster más cercano con una función de distancia *euclidean_dist*. Luego se guardan esos valores de los clusters y se procede a calcular los nuevos centroides por medio de la función *get_centroids*. Al tener los viejos y los nuevos centroides se pueden comparar por medio de la función *is_converged* y si no hay movimiento significativo, se termina el método. En caso de que continúen cambiando, se termina por límite de iteraciones.

1. *Método del Codo*

Otra parte importante de la clase del método es la idea de que también incluye un método para obtener la métrica de SSE, que es la importante para el método del codo. De este modo, después de ejecutar una clusterización se puede también acceder directamente a la métrica de errores y graficar estos datos después de juntarlos para varios números de clusters. Esto se hace con la función *_get_sse*. El SSE viene de la siguiente fórmula:

$$SSE = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

donde K es la cantidad de clusters, C_k es un conjunto que representa el cluster k, x_i es un punto y μ_k se refiere a la media del cluster.

Se ejecutó este método para cada una de las posibles cantidades de clusters para la base de datos desde 2 al 15, obteniendo los siguientes resultados:

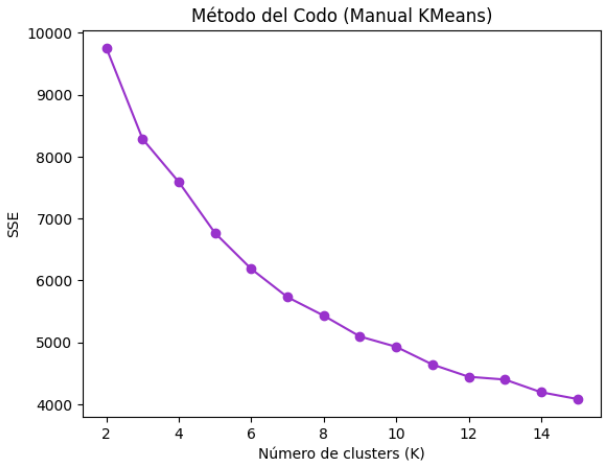

Figura 1: Gráfica de número de clústeres contra valor SSE. Método del codo realizado manualmente. Imagen de Autoría Propia

Según los resultados de la imagen, las diferencias no son tan grandes entre cantidad de clusters pero las más marcadas están al rededor de $k = 5$, por lo que eso podría ser una interesante referencia. La selección final dependerá del resultado del coeficiente de silueta también.

2. *Coeficiente de la Silueta*

A diferencia del método del codo, que evalúa la compacidad interna de los grupos, el método de la silueta ofrece una perspectiva complementaria al medir qué tan bien separado está cada punto respecto a su propio clúster en comparación con los demás. Formalmente, el coeficiente de silueta para un punto i se define como

$$s_i = \frac{b_i - a_i}{max(a_i, b_i)} \tag{1}$$

donde a_i representa la distancia promedio del punto hacia los demás elementos de su mismo clúster y b_i es la distancia promedio hacia los puntos del cluster vecino más cercano. El coeficiente resultante toma valores en el intervalo de -1 a 1, donde 1 indica que el punto está bien ubicado en su agrupación.

Se implementó de forma manual la función *silhouette_score_manual* que recorre cada punto del dataset, calculando s_i manualmente. Esta función se evaluó para valores k entre 2 a 15, obteniendo el coeficiente S promedio para cada uno.

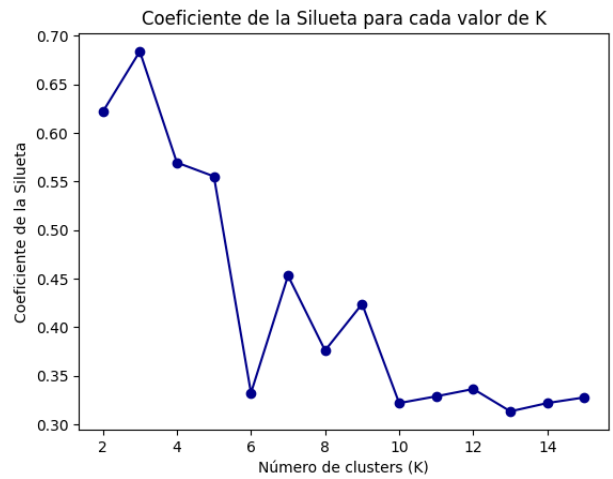

Figura 2: Gráfica de número de clústeres contra coeficiente de la silueta manualmente. Imagen de Autoría Propia

Los resultados indican que el valor de $k = 3$ produce el coeficiente más alto, con un valor aproximado de 0.68, lo que sugiere una separación razonablemente buena entre los grupos formados. Por este motivo, se selecciona $k = 3$ como el número óptimo de clusters según este criterio.

C. Comparación Scikit-Learn

Para la implementación con scikit-learn, se hizo uso de la clase KMeans directamente, guardando el SSE de cada valor de k desde 2 a 15. Después, se presenta el método del codo manual y el creado por scikit-learn en la misma gráfica a continuación:

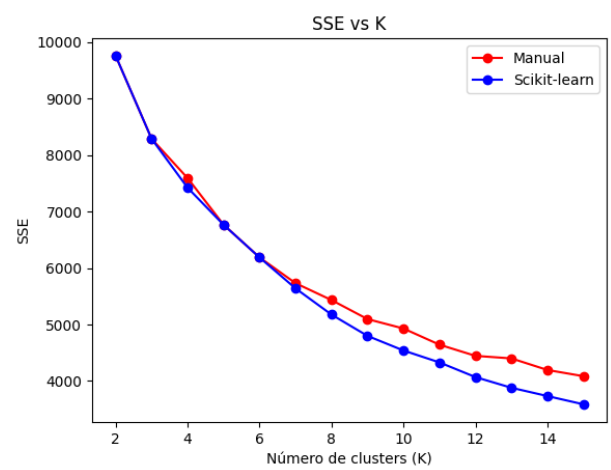

Figura 3: Gráfica de número de clústeres contra SSE para ambos modelos manual y el de scikit-learn. Autoría propia.

Se visualiza cómo el SSE para $k = 2, 3, 5, 6$ es extremadamente similar, pero a partir de $k = 7$ empiezan a alejarse entre sí conforme k aumenta. Para el modelo

de scikit, el codo es mucho más difícil de identificar, pues no se presenta una diferencia significativa visual entre los valores del SSE entre cada k consecutiva. Esto puede darse por una estructura débil en los datos, de tal forma que no existe una agrupación clave inmediata, o los clusters se encuentran superpuestos y mezclados.

A continuación, se calcula con la función `silhouette_score` de scikit-learn el cálculo del coeficiente silueta para el modelo de la librería. Se muestra abajo la gráfica correspondiente a la comparación entre el modelo manual y el de scikit-learn:

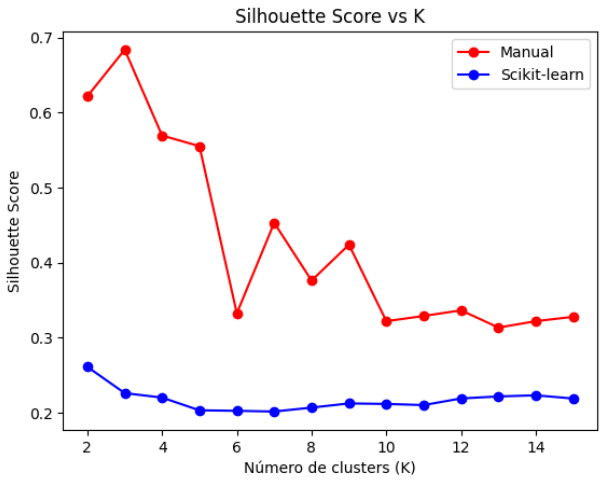

Figura 4: Gráfica de número de clústeres contra el coeficiente Silueta para ambos modelos manual y el de scikit-learn. Autoría propia.

En este gráfico sin embargo se presenta una mayor diferencia entre el modelo manual y el de scikit-learn. Esta variación se presenta inicialmente por la posible distribución inicial de los centroides: el modelo manual comienza el aleatorio con la función `random` de la librería NumPy, mientras que scikit-learn tiene su propia inicialización aleatoria de los clusters. Ambas son correctas, pero como bien se conoce, K-Means es un algoritmo en el que los centroides iniciales asignados pueden influír en el desempeño del modelo.

Para el modelo manual, se identifica claramente como $k = 3$ resulta ser aquel número con un mejor coeficiente silueta, mientras que para el modelo de scikit-learn, no se identifica un pico significativo más allá de $k = 2$, que aún así se encuentra abajo de 0.3, por lo que se indica que para este modelo no se discrimina de manera adecuada entre clusteres.

D. Implementación de PCA

Dado que el conjunto de features seleccionado cuenta con 6 dimensiones, no es posible visualizar directamente los clusters resultantes del modelo. Para resolver esto se recurre al Análisis de Componentes Principales (PCA), una técnica de reducción de dimensionalidad que construye nuevas variables como combinaciones lineales de las features originales. Cada componente se orienta en la dirección de máxima varianza del espacio, garantizando ortogonalidad entre sí, de modo que se preserva la mayor cantidad de información posible al reducir las dimensiones.

Para este trabajo se obtuvieron dos proyecciones: una de 2 componentes (PC1 y PC2) y otra de 3 componentes (PC1, PC2 y PC3), ambas calculadas con la implementación de *sklearn.decomposition.PCA* sobre los datos ya estandarizados. La proyección en 2D captura una porción representativa de la varianza total, suficiente para identificar visualmente la separación entre grupos. La proyección en 3D retiene una proporción aún mayor de la información original, permitiendo explorar con más detalle la estructura de los clusters.

Las visualizaciones se generaron tanto para el modelo manual como para el de scikit-learn, usando $k = 3$ como valor óptimo determinado por el coeficiente de silueta. En ambas proyecciones se empleó un color distinto por cluster y un marcador diferenciado según el true label de nivel educativo (círculo para **Higher Education**, cuadrado para **Low Education** y rombo para **Postgraduate**), lo que permite contrastar simultáneamente la segmentación obtenida por el modelo frente a la estructura real de los datos.

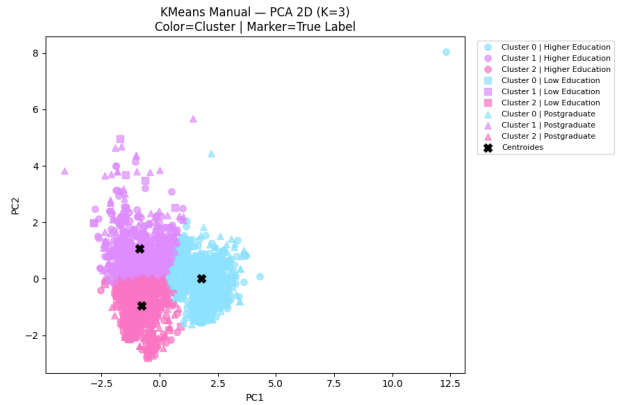

Figura 5: Segmentación de 3 clústeres con PCA para 2-dimensionalidad, añadiendo true labels. Imagen de Autoría Propia

En las gráficas 2D se aprecia que los clusters no coinciden de forma perfecta con los true labels, lo cual es un comportamiento esperado, dado que K-means agrupa por similitud en las variables numéricas de consumo, ingreso y demografía, mientras que el nivel educativo pudiera responder a una lógica distinta. La mezcla de marcadores dentro de un mismo color señala

en qué medida la segmentación económica del modelo se superpone con la clasificación educativa.

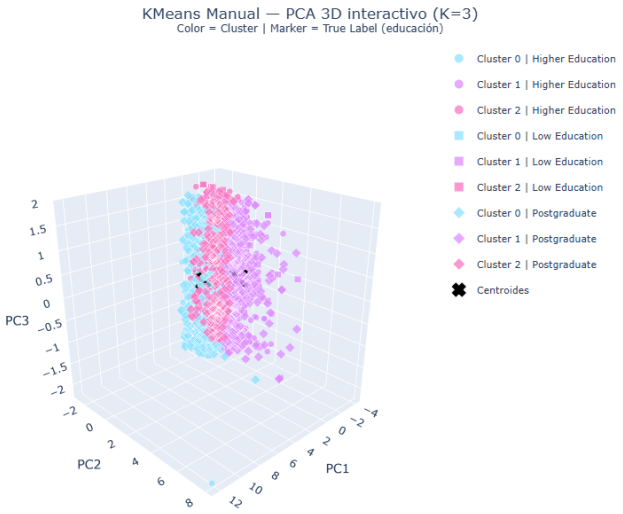

Figura 6: Segmentación de 3 clústeres con PCA para 3-dimensionalidad, añadiendo true labels. Imagen de Autoría Propia

La visualización 3D, generada de forma interactiva con *plotly*, confirma esta misma estructura añadiendo profundidad al análisis y evidenciando que ambos modelos producen una distribución de puntos prácticamente equivalente, con diferencias menores únicamente en puntos ubicados en las fronteras entre clusters.

1. *Resultados de Scikit-Learn*

Se muestran los resultados de la reducción de dimensionalidad producidos por las librerías de Python.

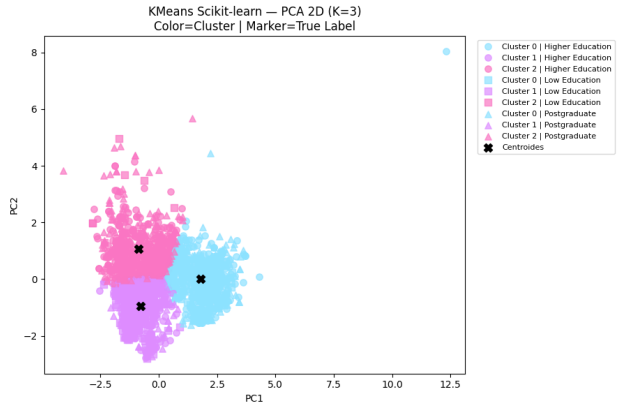

Figura 7: Segmentación de 3 clústeres con PCA para 2-dimensionalidad, añadiendo true labels. Imagen de Autoría Propia

Para 3 dimensiones:

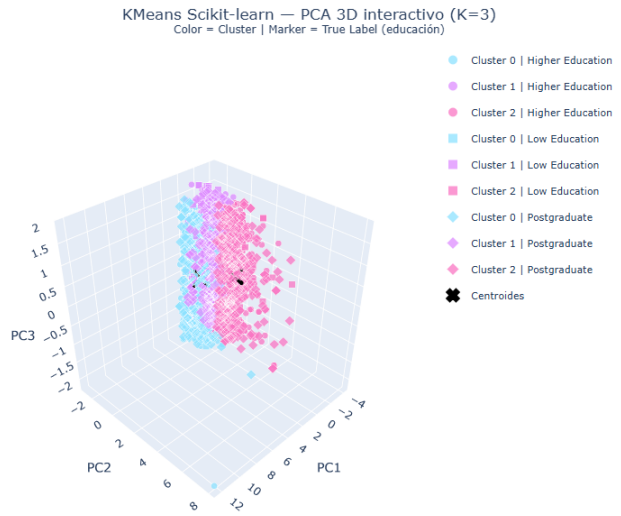

Figura 8: Segmentación de 3 clústeres con PCA para 3-dimensionalidad, añadiendo true labels. Imagen de Autoría Propia

Los resultados son prácticamente indiferentes.

E. Experimentos

Con el objetivo de determinar el número óptimo de clusters, se entrenaron modelos de K-means para valores de k desde 2 hasta 15, tanto con la implementación manual como con la de scikit-learn. Este rango se seleccionó para explorar un rango suficientemente amplio de configuraciones sin excederse en un costo computacional alto, considerando que valores muy altos de k tienden a producir clusters poco interpretables en un contexto de segmentación de clientes.

Para cada valor de k en el modelo manual se ejecutaron hasta 100 iteraciones, con inicialización aleatoria de centroides y criterio de convergencia basado en un umbral de 1×10^{-6} sobre el desplazamiento acumulado de los centroides entre iteraciones consecutivas. En cada corrida se registró el SSE correspondiente mediante la función `get_sse`, y se calculó el coeficiente de silueta con la implementación manual `silhouette_score_manual`. Para el modelo de scikit-learn se utilizaron los parámetros `random_state=42` y `n_init=10`, buscando reducir la sensibilidad a la inicialización aleatoria.

Una diferencia notable entre ambas implementaciones fue el tiempo de ejecución, donde el modelo manual requirió aproximadamente un minuto completo para recorrer todos los valores de k, mientras que scikit-learn completó el mismo proceso en pocos segundos.

Los resultados del método del codo mostraron que la inercia disminuye de forma consistente conforme aumenta k en ambos modelos, con una tasa de decremento similar hasta aproximadamente $k = 7$, punto a partir del cual las curvas comienzan a separarse. Por su parte, el coeficiente de silueta del modelo manual presentó un pico claro en $k = 3$ con un valor aproximado de 0.68, indicando una separación razonablemente buena entre los grupos. En contraste, el modelo de scikit-learn alcanzó su valor máximo en $k = 2$, con un coeficiente de apenas 0.26, lo que sugiere que bajo esta implementación la separación entre clusters es mucho menos definida.

V. DISCUSIÓN

Se observan diferencias significativas en los resultados del proceso de clusterización entre el modelo implementado manualmente y el proporcionado por la librería scikit-learn. Una de las principales causas de esta discrepancia radica en el método de inicialización de los centroides. Mientras que en la implementación manual es probable que se utilice una inicialización completamente aleatoria, scikit-learn emplea por defecto el método *k-means++*, el cual busca seleccionar centroides iniciales más separados entre sí para mejorar la convergencia y evitar soluciones pobres. Esta diferencia puede llevar a particiones distintas del espacio de datos, sin que esto implique que alguno de los modelos sea inválido. Además, es importante considerar que el algoritmo de K-Means es sensible a la inicialización y puede converger a mínimos locales. Por lo tanto, si el modelo manual no ejecuta múltiples inicializaciones (como sí lo hace scikit-learn mediante el parámetro `n_init`), es posible que haya encontrado una solución particularmente favorable en términos del coeficiente de silueta, pero no necesariamente representativa del comportamiento general del algoritmo.

Mediante la evaluacíon de SSE, no se identificaron diferencias extremadamente significativas; ambos modelos tenían valores similares hasta aproximadamente $k = 7$, donde empezaban a bifurcar ligeramente más conforme aumentaba k. La pendiente visualizada en el modelo manual es menos pronunciada que en el modelo de scikit-learn, pues empieza a .ªplanarse"poco a poco, pero aún disminuyendo.

En cuanto a la evaluación mediante el coeficiente de silueta, se identifican diferencias notables entre ambos enfoques. Para el modelo manual, el valor máximo se alcanza en $k = 3$ con un coeficiente de $s \approx 0{,}68$, lo cual sugiere una estructura de clusters bien definida y separada. En contraste, el modelo de *scikit-learn* presenta valores considerablemente más bajos, con un máximo aproximado de $s \approx 0{,}26$ en $k = 2$, y valores cercanos a 0,22 para $k = 3$. Esto indica que, bajo este modelo, los clusters son menos compactos y más solapados.

Esta discrepancia puede interpretarse de varias maneras. Por un lado, el modelo manual podría estar sobreajustándose a una configuración específica de centroides que maximiza artificialmente la separación entre clusters. Por otro lado, el modelo de *scikit-learn*, al utilizar múltiples inicializaciones y una estrategia más robusta, podría estar proporcionando una estimación más conservadora y estable de la estructura real de los datos.

Con respecto al análisis de componentes por partes (PCA), las proyecciones en 2 y 3 dimensiones permiten comprender a mayor profundidad el agrupamiento de los datos. Se realizó para ambos modelos manual y de scikit-learn, considerando como punto de partida las 6 dimensiones propuestas para el proyecto. Mediante las gráficas se puede deducir que la división de los clusters no coincide del todo con las etiquetas que se proponen para seccionar a los clientes por educación.

En la proyección a 2D mediante PCA, se observa la formación de una gran concentración de puntos en la región inferior izquierda del espacio (valores bajos de PC1 y PC2). Esta "nube"densa sugiere que múltiples observaciones, potencialmente de distintos grupos reales, están siendo proyectadas a una zona cercana, lo que dificulta la separación clara entre clusters en este espacio reducido. Este comportamiento es consistente con lo indicado por el coeficiente de silueta, el cual sugiere una separación moderada entre los clusters, evidenciando cierto traslape entre ellos. Cabe destacar que este patrón se presenta tanto en el modelo implementado manualmente como en el de la librería scikit-learn, lo cual sugiere que no es inherente del algoritmo, sino una característica de la estructura de los datos al ser proyectados en dos dimensiones.

En la proyección a 3D mediante PCA, los puntos se distribuyen formando una nube larga a lo largo del eje PC3, con solapamiento entre los tres sclusters. Si bien el tercer componente añade profundidad, la separación entre los grupos se mantiene leve, lo que es consist4ente con el coeficiente de silueta. Los centroides también se ubican relativamente cerca entre sí en el espacio, lo que refleja que las fronteras entre los clusters no son tan significativas. Similar a la proyección 2D,la implementación manual y de scikit-learn se mantienen extremadamente similares, lo que apunta nuevamente a que la yuxtaposición de clusters es una característica propia de los datos más allá de el algoritmo implementado.

VI. CONCLUSIONES

...A lo largo de este documento se buscó realizar una implementación manual del método de clustering K-means de la librería scikit-learn. Esto se planteó con

el uso de una base de datos de segmentación de clientes para comparar las diferencias entre el funcionamiento del método original de scikit y la simulación manual.

Para realizar la implementación manual correctamente se simuló la base matemática del método original por medio del proceso iterativo de crear centroides, asignar todos los puntos al centroide más cercano y recalcular los centroides en base a los nuevos grupos formados. También se realizó un análisis del método del codo elaborado manualmente con la métrica SSE y el coeficiente de silueta para determinar con mayor seguridad la cantidad de clústeres para segmentar la población. Además, se implementó un modelo de Análisis de Componentes Principales (PCA) para reducir la dimensionalidad de los datos y hacerlos visualizables por medio de gráficas.

A lo largo de este documento se buscó realizar una implementación manual del método de clustering K-means de la librería scikit-learn. Esto se planteó con el uso de una base de datos de segmentación de clientes para comparar las diferencias entre el funcionamiento del método original de scikit y la simulación manual.

Para realizar la implementación manual correctamente se simuló la base matemática del método original por medio del proceso iterativo de crear centroides, asignar todos los puntos al centroide más cercano y recalcular los centroides en base a los nuevos grupos formados. También se realizó un análisis del método del codo elaborado manualmente con la métrica SSE y el coeficiente de silueta para determinar con mayor seguridad la cantidad de clústeres para segmentar la población. Además, se implementó un modelo de Análisis de Componentes Principales (PCA) para reducir la dimensionalidad de los datos y hacerlos visualizables por medio de gráficas.

Los resultados de esta implementación mostraron una segmentación virtualmente idéntica a la original producida por la librería de scikit. No obstante, las diferencias están considerablemente más marcadas cuando se observan las gráficas del método del codo y el coeficiente de la silueta, ya que la ejecución del método del codo tiene inercias un poco más elevadas y la de las siluetas es significativamente más inestable que la ejecución original. Esto muestra que aunque la estructura de los métodos de clustering es muy similar, pequeñas diferencias en la selección de centroides originales y en la manera de generar los grupos puede producir cambios importantes en las métricas. Es posible que las diferencias entre los resultados de las métricas se deban a la selección de centroides originales, ya que scikit utiliza por defecto el método K-means++ para este proceso. Otra consideración importante es la amplia cantidad de optimizaciones numéricas y de estabilidad que la librería original ha refinado durante años.

En conclusión, la implementación manual de K-means permitió comprender a profundidad la lógica y fundamentos matemáticos detrás del algoritmo, evidenciando que, aunque es posible replicar resultados similares a los de scikit-learn, las diferencias en inicialización y optimización influyen de manera notable en las métricas de evaluación. Esto resalta la importancia tanto de los detalles técnicos en la implementación como del uso de herramientas consolidadas para obtener resultados más estables y eficientes en problemas reales de segmentación.

[1] J. Prosise, *Applied Machine Learning and AI for Engineers: Solve Business Problems That Can't Be Solved Algorithmically* (O'Reilly Media, 2022).

[2] A. C. Müller and S. Guido, *Introduction to Machine Learning with Python: A Guide for Data Scientists* (O'Reilly Media, Inc., 2016).

[3] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, and E. Duchesnay, Scikit-learn: Machine learning in python, Journal of Machine Learning Research **12**, 2825 (2011).

[4] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. (Pearson, 2020).